Month-End Variance Investigator

Ayaan Khan | MiraFlores case study | Prepared for GreenGrowth CPAs

Run the prototype: https://colab.research.google.com/github/ayaan-27/Miraflores_Project/blob/main/notebooks/Month_end_review.ipynb

Select Runtime → Run all. The notebook starts the Streamlit app and prints a temporary public link that remains available while its runtime is running.

Source and setup: https://github.com/ayaan-27/Miraflores_Project

1. The problem I chose and what motivated it

I considered three problems connected to work I have done.

Understanding a financial movement across records. I have worked on text-to-SQL agents that retrieve information across relational databases. That motivated my interest in what happens after a number is retrieved: a reviewer still needs to understand the movement, establish its supporting evidence, and decide whether any action is needed. I chose month-end expense variances as a concrete setting for that problem. An unexplained movement can lead to repeated queries, incomplete explanations, or a delayed close.

Connecting requests to useful evidence. At Ricoh, I worked on document retrieval that preserved references between service instructions and their supporting diagrams. That motivated a second problem: matching an audit request to a document that covers the right entity, period, and requirement. A file being uploaded does not establish that it resolves a request. Missing or unsuitable support can create repeated follow-ups.

Preparing evaluation data without exposing sensitive information. At Mphasis, I worked on synthetic-data generation and sensitive-information redaction. Those projects motivated my interest in representative datasets that teams can use without uncontrolled disclosure. Restricted access can slow evaluation, while poorly controlled sharing creates privacy risks.

I chose variance investigation because it combines structured queries, documents, calculations, and a review decision in one narrow workflow. It also lets me demonstrate an important distinction: a large movement can be legitimate, or it can indicate a treatment that deserves review. I have not interviewed GreenGrowth's staff or observed its close process. The frequency and cost of this problem in that firm remain assumptions.

2. How I thought about the approach

My starting question was: what would let a reviewer move from “this account changed” to a defensible next step? A generated explanation alone would be insufficient. I wanted a reproducible connection between the account movement, the underlying posting, its source document, the relevant policy, and the reviewer decision.

I assumed the current process involves comparing account balances, drilling into a general ledger, locating invoices or agreements, and recording an explanation in a spreadsheet or close workpaper. I would validate that assumption before building integrations. A better outcome

would be less repeated searching and clearer support for a conclusion, without reducing the reviewer's control.

I broke the first step into four stages: detect the movement, investigate with evidence tools, independently check the treatment, and pause for review. Missing evidence should return to the same case when it arrives. That requirement shaped the state and checkpoint design: the previous request must remain visible, and new support must cause the calculations to run again.

Why LangGraph. Its explicit state, conditional execution, tool loop, and human interrupts fit this sequence. The model chooses read-only queries after observing earlier results. Code requires the necessary evidence coverage and limits the number of decisions. A person can request a document; supplying it resumes the saved graph thread. This gives me an execution structure that can be inspected and tested.

Where I used conventional software. CSV validation, SQL queries, labeled PDF fields, monetary calculations, service-period allocation, balanced draft journals, and permitted review actions are implemented in code. The model helps choose evidence queries and draft a short cited explanation. It cannot change an amount, invent an accepted source, or approve or post an entry.

A credible alternative. A spreadsheet with materiality formulas, saved ledger filters, and a standard checklist could be simpler and cheaper. That is my baseline. I chose the graph for this prototype because it can connect query results to document checks, retain a pending request, and resume after new evidence. If that loop does not reduce total reviewer effort, I would use the simpler process or extend an existing platform.

The first scope covers one positive posting that reconciles a material expense movement in a two-month close. It uses synthetic records and a supplied illustrative accrual policy. The broader opportunity is an evidence-supported close workspace connected to actual accounting systems, subject to validated demand and stronger access and storage controls.

3. The working prototype

Stage 1: Detect. The Streamlit app validates a CSV expense-ledger extract and loads it into SQLite. It calculates September and October balances and selects movements meeting an illustrative $5,000 and 20% threshold. Duplicate transaction IDs, invalid amounts, unsupported currencies, and out-of-scope dates are rejected before replacing the workspace.

Stage 2: Investigate. A local model chooses tools for ledger detail, vendor totals, the posting document, supporting agreement, and close policy. Tools execute parameterized, read-only queries or read actual PDFs. The activity record contains the selected actions, returned facts, and source references. PDF fields use an explicit labeled format and preserve page numbers and exact source lines. General document extraction and scanned-image OCR are deferred.

Stage 3: Check. Independent code tests posting uniqueness, the variance bridge, source consistency, the supported accounting policy, and the service period or bonus support. Allocation uses integer cents and preserves the contract total when rounding. Partial months, prior-period effects, multiple drivers, and inconsistent sources remain unresolved. A proposed

journal must balance. The model explanation is checked for source IDs and unsupported figures or action claims; rejected text is not presented as an accepted explanation.

Stage 4: Review and resume. The graph saves a SQLite checkpoint before review. The reviewer can request evidence, approve a supported workpaper, or record a hold. A supplied document must match the expected ID, entity, vendor, invoice, amount, currency, and required fields before it can change the case. Invalid evidence leaves the prior checkpoint intact. Approved workpapers still contain draft entries; no financial action or email is sent.

Representative case. Software expense rises from $2,000 to $26,000. Queries identify an additional $24,000 renewal, but its invoice does not establish the service dates. The case requests the missing agreement. The reviewer supplies a contract covering October through the following September. Under the supplied policy, $2,000 belongs to October and $22,000 to later periods. The application proposes a $22,000 debit to prepaid services and credit to software expense. October expense after that proposed adjustment is $4,000, including the original recurring $2,000. A new review decision is required.

Other and difficult cases. An approved October bonus supports an $18,000 payroll movement without an adjustment. A current-month consulting agreement supports another movement without a deferral. A wrong contract and malformed ledger are rejected while preserving the existing case. Outputs include a PDF workpaper, proposed-journal CSV, and JSON review record with sources, calculations, decisions, and activity.

The app executes the actual graph and local model on its server without an inference API key. The company, ledger, policy, and documents are synthetic; accounting-system integrations are not implemented. Source consistency does not prove authenticity. Reviewer names are entered rather than authenticated, and temporary hosting storage is not an immutable or durable audit record.

4. Validation, learning, and disclosure

Twelve workflow, accounting, persistence, and Streamlit widget tests pass with an explicitly injected test planner. They exercise the actual graph, tools, PDFs, calculations, approval restrictions, invalid evidence, restart recovery on retained disk, and retries. Separate real-model validation completed the software evidence loop and payroll case with 21 actual model calls. It produced the expected $22,000 proposal and $4,000 adjusted expense, and proposed no payroll adjustment. These small synthetic checks do not establish accuracy on unseen client records. A separate browser test also passed against the actual Streamlit app with real model inference, including wrong/correct PDF uploads, saved-case reload, invalid CSV rejection, and PDF/CSV/JSON exports. The measured app process peaked at about 939 MiB in that test; hosted performance still needs confirmation.

I would next observe three close reviewers working with consented, redacted examples. I want to learn whether the bottleneck is retrieving transactions, locating documents, interpreting treatment, or waiting for replies. I would compare this workflow with the spreadsheet baseline on five to ten willing datasets, measuring total review time, unsupported explanations, incorrect evidence assignments, and reopened cases. A provisional goal is 25% less reviewer time without increasing incorrect recommendations; that is a target to test, not an achieved saving.

I would simplify or stop if existing tools already solve the problem, preparing the inputs costs more time than the workflow saves, or model output creates excessive checking. Validated demand would justify broader extraction, actual integrations, authenticated identities, controlled durable storage, and evaluation across more accounting situations.

I used an AI coding assistant for scoping, implementation, testing, and drafting. I checked its output through exact expected calculations, invalid inputs, executable workflow tests, and actual model runs. Local-inference and document-export patterns were adapted from an earlier prototype; no employer code or client data was used. The variance data, controls, graph, and Streamlit interface were built for this scope. The core application took approximately one hour of AI-assisted implementation and validation, including automated setup and model checks. Notebook launch packaging, earlier prototype experiments, and my personal review/video preparation are separate.